

Experiment No. 1

Objective:

Implement Breadth First Search (BFS) for a given graph.

Algorithm (BFS):

1. Start from a given source node.
2. Initialize a queue and add the source node to it.
3. Mark the source node as visited.
4. While the queue is not empty:
 - Dequeue a node from the front.
 - Visit and process the node.
 - Enqueue all unvisited adjacent nodes and mark them as visited.
5. Repeat until all reachable nodes are visited.

Python Code:

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

nodes = int(input("Enter number of nodes: "))
edges = int(input("Enter number of edges: "))

graph = {i: [] for i in range(nodes)}

for _ in range(edges):
    u, v = map(int, input("Enter edge (u v): ").split())
    graph[u].append(v)
    graph[v].append(u)

start_node = int(input("Enter starting node: "))
bfs(graph, start_node)
```

Sample Input:

```
Enter number of nodes: 5
Enter number of edges: 4
Enter edge (u v): 0 1
Enter edge (u v): 0 2
```

Enter edge (u v): 1 3
Enter edge (u v): 1 4
Enter starting node: 0

Sample Output:

0 1 2 3 4

Objective:

Implement Breadth First Search (BFS) to traverse a given maze and find a path from the start to the goal.

Algorithm (BFS for Maze):

1. Represent the maze as a 2D grid where 0 is a free path and 1 is a wall.
2. Start from the source cell (start position).
3. Initialize a queue and add the start cell to it.
4. Maintain a visited set to avoid revisiting cells.
5. While the queue is not empty:
 - Dequeue a cell from the front.
 - If it is the goal cell, stop and return the path.
 - For each valid adjacent cell (up, down, left, right):
 - If the cell is free and unvisited, mark it visited and enqueue it.
6. If the goal is reached, print the path; otherwise, report no path found.

Python Code:

```
from collections import deque
```

```
def bfs_maze(maze, start, goal):
    rows, cols = len(maze), len(maze[0])
    queue = deque([(start, [start])])
    visited = set([start])

    directions = [(-1,0),(1,0),(0,-1),(0,1)]

    while queue:
        (r, c), path = queue.popleft()
        if (r, c) == goal:
            return path
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc] == 0 and (nr,nc) not in visited:
                visited.add((nr,nc))
                queue.append(((nr,nc), path + [(nr,nc)]))
    return None
```

```
rows = int(input("Enter number of rows in maze: "))
cols = int(input("Enter number of columns in maze: "))
```

```
maze = []
print("Enter maze row by row (0 for free, 1 for wall):")
for _ in range(rows):
    maze.append(list(map(int, input().split())))
```

```
start_row = int(input("Enter start row: "))
start_col = int(input("Enter start column: "))
goal_row = int(input("Enter goal row: "))
goal_col = int(input("Enter goal column: "))
```

```
path = bfs_maze(maze, (start_row, start_col), (goal_row, goal_col))
```

```
if path:
    print("Path found:")
    for cell in path:
        print(cell, end=" ")
else:
    print("No path found")
```

Sample Input:

Enter number of rows in maze: 4
Enter number of columns in maze: 4
Enter maze row by row (0 for free, 1 for wall):
0 1 0 0
0 0 0 1
1 0 1 0
0 0 0 0
Enter start row: 0
Enter start column: 0
Enter goal row: 3
Enter goal column: 3

Sample Output:

Path found:
(0, 0) (1, 0) (1, 1) (1, 2) (2, 2) (3, 2) (3, 3)

Experiment No. 2

Objective:

Implement Depth First Search (DFS) for a tree or graph structure.

Algorithm (DFS):

1. Start from a given source node.
2. Initialize a set or list to keep track of visited nodes.
3. Visit the current node and mark it as visited.
4. Recursively visit all unvisited adjacent nodes.
5. Repeat until all reachable nodes from the source are visited.
6. For a tree, the process is similar but without cycles, so visited tracking is optional.

Python Code:

```
def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()
    visited.add(node)
    print(node, end=" ")
    for neighbor in graph.get(node, []):
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

nodes = int(input("Enter number of nodes: "))
edges = int(input("Enter number of edges: "))

graph = {i: [] for i in range(nodes)}

for _ in range(edges):
    u, v = map(int, input("Enter edge (u v): ").split())
    graph[u].append(v)
    graph[v].append(u)

start_node = int(input("Enter starting node: "))
dfs(graph, start_node)
```

Sample Input:

```
Enter number of nodes: 5
Enter number of edges: 4
Enter edge (u v): 0 1
Enter edge (u v): 0 2
Enter edge (u v): 1 3
Enter edge (u v): 1 4
Enter starting node: 0
```

Sample Output:

```
0 1 3 4 2
```

Experiment No. 3

Objective:

Solve the 8-Puzzle Problem using A* Search Algorithm.

Algorithm (A* Search for 8-Puzzle):

1. Represent the puzzle as a 3x3 grid (or a list of 9 elements).
2. Define the goal state (usually [1,2,3,4,5,6,7,8,0], where 0 is the empty tile).
3. Initialize a priority queue (open list) with the start state, using $f(n) = g(n) + h(n)$, where:
 - $g(n)$ = cost to reach current state
 - $h(n)$ = heuristic estimate to goal (e.g., Manhattan distance)
4. Initialize a closed list to store visited states.
5. While the open list is not empty:
 - Pop the state with the lowest $f(n)$ from the queue.
 - If it is the goal state, return the solution path.
 - Generate all valid moves (up, down, left, right).
 - For each move, calculate $g(n)$ and $h(n)$, and add new states to the open list if not visited.
6. Repeat until goal state is reached or no solution exists.

Python Code:

```
import heapq

def manhattan_distance(state, goal):
    distance = 0
    for i in range(9):
        if state[i] != 0:
            goal_index = goal.index(state[i])
            distance += abs(i // 3 - goal_index // 3) + abs(i % 3 - goal_index % 3)
    return distance

def astar(start, goal):
    open_list = []
    heapq.heappush(open_list, (manhattan_distance(start, goal), 0, start, []))
    visited = set()

    while open_list:
        f, g, current, path = heapq.heappop(open_list)
        if current == goal:
            return path + [current]
        visited.add(tuple(current))

        zero_index = current.index(0)
        moves = []
        if zero_index % 3 > 0: moves.append(-1)
        if zero_index % 3 < 2: moves.append(1)
        if zero_index // 3 > 0: moves.append(-3)
        if zero_index // 3 < 2: moves.append(3)
```

```

        for move in moves:
            new_index = zero_index + move
            new_state = current[:]
            new_state[zero_index], new_state[new_index] = new_state[new_index],
new_state[zero_index]
            if tuple(new_state) not in visited:
                heapq.heappush(open_list, (g + 1 + manhattan_distance(new_state, goal), g + 1,
new_state, path + [current]))
        return None

```

```

print("Enter start state (9 numbers, 0 is empty tile, space separated):")
start_state = list(map(int, input().split()))
goal_state = [1,2,3,4,5,6,7,8,0]

```

```

solution = astar(start_state, goal_state)
if solution:
    print("Solution found in {} moves:".format(len(solution)-1))
    for state in solution:
        for i in range(0,9,3):
            print(state[i:i+3])
        print()
else:
    print("No solution found.")

```

Sample Input:

Enter start state (9 numbers, 0 is empty tile, space separated):
1 2 3 4 0 6 7 5 8

Sample Output:

Solution found in 3 moves:

```

[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

```

```

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

```

```

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

```

Experiment No. 4

Objective:

Implement Hill Climbing Algorithm for numerical optimization.

Algorithm (Hill Climbing for Numerical Optimization):

1. Choose an initial solution (starting value).
2. Evaluate neighboring solutions by making small changes to the current solution.
3. Move to the neighbor with the highest value.
4. Repeat steps 2-3 until no neighbor has a higher value than the current solution (local maximum reached).
5. Return the current solution and its function value.

Python Code:

```
def hill_climb(func, x_start, step_size=0.1, max_iterations=1000):
    current = x_start
    for _ in range(max_iterations):
        neighbors = [current + step_size, current - step_size]
        next_move = max(neighbors, key=func)
        if func(next_move) <= func(current):
            break
        current = next_move
    return current, func(current)

def func(x):
    return -(x-3)**2 + 9 # Example function with maximum at x=3

start = float(input("Enter starting value: "))
solution, value = hill_climb(func, start)
print("Best solution:", solution)
print("Function value at best solution:", value)
```

Sample Input:

Enter starting value: 0

Sample Output:

Best solution: 3.0

Function value at best solution: 9.0

Objective:

Implement Hill Climbing Algorithm for numerical pathfinding.

Algorithm (Hill Climbing for Pathfinding):

1. Choose a starting position.
2. Evaluate all neighboring positions (possible moves).
3. Move to the neighbor with the lowest heuristic value (closest to the goal).
4. Repeat steps 2-3 until the goal is reached or no neighbor improves the heuristic (local optimum).
5. Return the path taken from start to goal.

Python Code:

```
def hill_climb_path(start, goal, heuristic):
    current = start
    path = [current]
    while current != goal:
        neighbors = [current - 1, current + 1] # Example: 1D moves
        next_move = min(neighbors, key=heuristic)
        if heuristic(next_move) >= heuristic(current):
            break
        current = next_move
        path.append(current)
    return path

def heuristic(x):
    return abs(x - 10) # Goal at position 10

start_pos = int(input("Enter start position: "))
path = hill_climb_path(start_pos, 10, heuristic)
print("Path:", path)
```

Sample Input:

Enter start position: 0

Sample Output:

Path: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Experiment No. 5

Objective:

Implement Simulated Annealing Algorithm for constraint-based search problems.

Algorithm (Simulated Annealing):

1. Start with an initial solution and initial temperature T .
2. Evaluate the cost (or energy) of the current solution.
3. While temperature T is above a minimum threshold:
 - Generate a neighboring solution by making a small change to the current solution.
 - Calculate the cost difference $\Delta E = \text{cost}(\text{neighbor}) - \text{cost}(\text{current})$.
 - If $\Delta E < 0$ (neighbor is better), move to the neighbor.
 - Else, move to the neighbor with probability $e^{(-\Delta E/T)}$.
 - Decrease temperature T according to a cooling schedule.
4. Repeat until temperature is very low or an acceptable solution is found.
5. Return the best solution found.

Python Code:

```
import math
import random

def simulated_annealing(cost_func, initial_solution, initial_temp=1000, cooling_rate=0.95,
min_temp=0.01, max_iterations=1000):
    current = initial_solution
    current_cost = cost_func(current)
    best = current
    best_cost = current_cost
    T = initial_temp

    for _ in range(max_iterations):
        if T < min_temp:
            break
        neighbor = current + random.choice([-1, 1]) # Example: small change
        neighbor_cost = cost_func(neighbor)
        delta = neighbor_cost - current_cost

        if delta < 0 or random.random() < math.exp(-delta / T):
            current = neighbor
            current_cost = neighbor_cost
            if current_cost < best_cost:
                best = current
                best_cost = current_cost
            T *= cooling_rate

    return best, best_cost

def cost(x):
    return (x-5)**2 # Example: minimum at x=5

start = float(input("Enter initial solution: "))
```

```
solution, value = simulated_annealing(cost, start)
print("Best solution:", solution)
print("Cost at best solution:", value)
```

Sample Input:

Enter initial solution: 0

Sample Output:

Best solution: 5.0

Cost at best solution: 0.0

Experiment No. 6

Objective:

Solve the Water Jug Problem using state-space search.

Algorithm (BFS for Water Jug Problem):

1. Represent each state as a tuple (jug1, jug2).
2. Initialize a queue with the initial state and a set to track visited states.
3. While the queue is not empty:
 - Dequeue the front state.
 - If the goal quantity is reached in any jug, return the path.
 - Generate all possible next states from the current state:
 - a. Fill jug1 or jug2.
 - b. Empty jug1 or jug2.
 - c. Pour water from jug1 to jug2 and vice versa.
 - Add new states to the queue if they are not visited.
4. Repeat until the goal is reached or no solution exists.

Python Code:

```
from collections import deque
```

```
def bfs_water_jug(capacity1, capacity2, goal):
```

```
    start = (0, 0)
```

```
    queue = deque([(start, [start])])
```

```
    visited = set([start])
```

```
    while queue:
```

```
        (jug1, jug2), path = queue.popleft()
```

```
        if jug1 == goal or jug2 == goal:
```

```
            return path
```

```
        states = set()
```

```
        states.add((capacity1, jug2)) # Fill jug1
```

```
        states.add((jug1, capacity2)) # Fill jug2
```

```
        states.add((0, jug2)) # Empty jug1
```

```
        states.add((jug1, 0)) # Empty jug2
```

```
        # Pour jug1 -> jug2
```

```
        pour = min(jug1, capacity2 - jug2)
```

```
        states.add((jug1 - pour, jug2 + pour))
```

```
        # Pour jug2 -> jug1
```

```
        pour = min(jug2, capacity1 - jug1)
```

```
        states.add((jug1 + pour, jug2 - pour))
```

```
        for state in states:
```

```
            if state not in visited:
```

```
                visited.add(state)
```

```
                queue.append((state, path + [state]))
```

```
    return None
```

```
capacity1 = int(input("Enter capacity of Jug 1: "))
capacity2 = int(input("Enter capacity of Jug 2: "))
goal = int(input("Enter goal amount: "))

solution = bfs_water_jug(capacity1, capacity2, goal)
if solution:
    print("Solution found:")
    for state in solution:
        print(state)
else:
    print("No solution found.")
```

Sample Input:

Enter capacity of Jug 1: 4
Enter capacity of Jug 2: 3
Enter goal amount: 2

Sample Output:

Solution found:

(0, 0)
(4, 0)
(1, 3)
(1, 0)
(0, 1)
(4, 1)
(2, 3)

Experiment No. 7

Objective:

Write Prolog programs to define family relationships using predicates.

Algorithm (Prolog Family Relationships):

1. Define basic facts: parent, male, female.
2. Define rules using facts:
 - father(X, Y): X is father of Y
 - mother(X, Y): X is mother of Y
 - sibling(X, Y): X and Y share at least one parent
 - grandparent(X, Y): X is parent of Y's parent
 - uncle(X, Y): X is brother of Y's parent
 - aunt(X, Y): X is sister of Y's parent
3. Query the database using these rules to find relationships.

Prolog Code:

```
% Facts
male(john).
male(mike).
male(tom).
female(susan).
female(linda).
female(anna).

parent(john, mike).
parent(john, linda).
parent(susan, mike).
parent(susan, linda).
parent(mike, tom).
parent(anna, tom).

% Rules
father(X, Y) :- male(X), parent(X, Y).
mother(X, Y) :- female(X), parent(X, Y).
sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
uncle(X, Y) :- male(X), sibling(X, Z), parent(Z, Y).
aunt(X, Y) :- female(X), sibling(X, Z), parent(Z, Y).
```

Sample Queries:

```
?- father(john, mike).
?- mother(susan, linda).
?- sibling(mike, linda).
?- grandparent(john, tom).
?- uncle(mike, tom).
?- aunt(linda, tom).
```

Expected Output:

true.

true.

true.

true.

false.

True.

Experiment No. 8

Objective:

Solve the 4-Queens Problem using backtracking in Prolog.

Algorithm (4-Queens Problem in Prolog):

1. Represent the solution as a list of column positions for each row.
2. Place a queen in the first row, then recursively place queens in subsequent rows.
3. For each queen placement, check:
 - No two queens are in the same column.
 - No two queens are on the same diagonal.
4. Backtrack if a placement leads to a conflict.
5. Return all solutions where 4 queens are placed safely.

Prolog Code:

```
% Define the 4-Queens problem
queens(Board) :-
    Board = [A,B,C,D],
    permutation([1,2,3,4], Board),
    safe(Board).

% Check safety of the board
safe([]).
safe([Q|Others]) :-
    safe(Others),
    no_attack(Q, Others, 1).

% Ensure Q does not attack any other queens
no_attack(_, [], _).
no_attack(Q, [Q1|Rest], Distance) :-
    Q \= Q1 + Distance,
    Q \= Q1 - Distance,
    NextDistance is Distance + 1,
    no_attack(Q, Rest, NextDistance).

% Example query:
% ?- queens(Board).
```

Sample Query:

```
?- queens(Board).
```

Expected Output:

```
Board = [2, 4, 1, 3] ;
Board = [3, 1, 4, 2] ;
false.
```


Experiment No. 9

Objective:

Implement Unification Algorithm in Python or Prolog.

Algorithm (Unification):

1. Represent expressions as variables, constants, or compound structures (tuples).
2. Initialize an empty substitution dictionary.
3. Compare the two expressions:
 - If both are identical constants, succeed.
 - If one is a variable, bind it to the other expression if it does not create a circular reference.
 - If both are compound structures, unify their corresponding components recursively.
4. If all components unify, return the substitution.
5. If any conflict occurs, fail unification.

Python Code:

```
def is_variable(x):
    return isinstance(x, str) and x.islower()

def unify(x, y, subst=None):
    if subst is None:
        subst = {}
    if x == y:
        return subst
    if is_variable(x):
        return unify_var(x, y, subst)
    if is_variable(y):
        return unify_var(y, x, subst)
    if isinstance(x, tuple) and isinstance(y, tuple) and len(x) == len(y):
        for xi, yi in zip(x, y):
            subst = unify(xi, yi, subst)
            if subst is None:
                return None
        return subst
    return None

def unify_var(var, x, subst):
    if var in subst:
        return unify(subst[var], x, subst)
    elif is_variable(x) and x in subst:
        return unify(var, subst[x], subst)
    else:
        subst[var] = x
        return subst

# Example usage
expr1 = ('f', 'x', 'y')
expr2 = ('f', 'a', 'b')
solution = unify(expr1, expr2)
print("Substitution:", solution)
```

Sample Input:

expr1 = ('f', 'x', 'y')

expr2 = ('f', 'a', 'b')

Sample Output:

Substitution: {'x': 'a', 'y': 'b'}

Experiment No. 10

Objective:

Implement Forward and Backward Chaining in a rule-based system (manual or code-based).

Algorithm (Forward Chaining):

1. Initialize the set of known facts.
2. Repeat:
 - For each rule, check if all its premises are in the known facts.
 - If yes, add the conclusion of the rule to the known facts.
3. Continue until no new facts can be inferred.

Algorithm (Backward Chaining):

1. Start with a query (goal) to be proven.
2. Check if the goal is already in the known facts.
3. If not, find rules whose conclusion matches the goal.
4. Recursively attempt to prove all premises of those rules.
5. If all premises of a rule are proven, the goal is proven.

Python Code (Forward and Backward Chaining):

```
# Knowledge base
rules = [
    (['rain'], 'wet_ground'),
    (['sprinkler'], 'wet_ground'),
    (['wet_ground', 'slippery'], 'accident'),
    (['wet_ground'], 'slippery')
]

facts = set(['rain'])

# Forward Chaining
def forward_chaining(rules, facts):
    inferred = True
    while inferred:
        inferred = False
        for premises, conclusion in rules:
            if all(p in facts for p in premises) and conclusion not in facts:
                facts.add(conclusion)
                inferred = True
    return facts

# Backward Chaining
def backward_chaining(goal, rules, facts):
    if goal in facts:
        return True
    for premises, conclusion in rules:
        if conclusion == goal:
            if all(backward_chaining(p, rules, facts) for p in premises):
                facts.add(goal)
                return True
```

```
return False
```

```
# Dynamic query input
```

```
query = input("Enter goal to prove: ")
```

```
# Run forward chaining
```

```
all_facts = forward_chaining(rules, facts.copy())
```

```
print("Facts inferred by Forward Chaining:", all_facts)
```

```
# Run backward chaining
```

```
result = backward_chaining(query, rules, facts.copy())
```

```
print(f"Backward Chaining: Goal '{query}' is", "proven" if result else "not proven")
```

Sample Input:

Enter goal to prove: accident

Sample Output:

Facts inferred by Forward Chaining: {'rain', 'wet_ground', 'slippery', 'accident'}

Backward Chaining: Goal 'accident' is proven

Experiment No. 11

Objective:

Demonstrate Resolution in Propositional Logic through a basic example (e.g., proving a theorem).

Algorithm (Resolution in Propositional Logic):

1. Express all statements (premises and negated goal) in Conjunctive Normal Form (CNF).
2. Represent each clause as a set of literals.
3. Repeat:
 - Pick two clauses that contain complementary literals.
 - Resolve them to produce a new clause.
 - If the empty clause is produced, the theorem is proven.
 - Add new clauses to the set of clauses.
4. If no new clauses can be produced and empty clause is not found, theorem is not provable.

Example:

Premises:

1. $P \rightarrow Q$ (equivalent to $\neg P \vee Q$)
2. $Q \rightarrow R$ (equivalent to $\neg Q \vee R$)

Goal: Prove $P \rightarrow R$

Convert goal to negation for refutation: $\neg(P \rightarrow R) = P \wedge \neg R$

CNF Clauses:

1. $\neg P \vee Q$
2. $\neg Q \vee R$
3. P
4. $\neg R$

Python Code (Resolution Example):

```
def resolve(ci, cj):
    resolvents = []
    for di in ci:
        for dj in cj:
            if di == f'~{dj}' or dj == f'~{di}':
                new_clause = set(ci) | set(cj)
                new_clause.discard(di)
                new_clause.discard(dj)
                resolvents.append(new_clause)
    return resolvents

clauses = [
    {'~P', 'Q'},
    {'~Q', 'R'},
    {'P'},
    {'~R'}
]

new = set()
```

```
while True:
    n = len(clauses)
    pairs = [(clauses[i], clauses[j]) for i in range(n) for j in range(i+1, n)]
    added = False
    for (ci, cj) in pairs:
        resolvents = resolve(ci, cj)
        for res in resolvents:
            if not res:
                print("Empty clause derived. Theorem is proven.")
                exit()
            if res not in clauses and res not in new:
                new.add(frozenset(res))
                added = True
    if not added:
        print("No empty clause derived. Theorem cannot be proven.")
        break
    for c in new:
        clauses.append(set(c))
    new.clear()
```

Sample Output:

Empty clause derived. Theorem is proven

Experiment No. 12

Objective:

Perform stemming and lemmatization on user-input text.

Algorithm:

1. Take input text from the user.
2. Tokenize the text into words.
3. Apply a stemming algorithm (e.g., PorterStemmer) to reduce words to their root forms.
4. Apply a lemmatization algorithm (e.g., WordNetLemmatizer) to reduce words to their dictionary base form.
5. Display the results of stemming and lemmatization.

Python Code:

```
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize

nltk.download('punkt')
nltk.download('wordnet')

text = input("Enter text: ")

words = word_tokenize(text)

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

stemmed_words = [stemmer.stem(word) for word in words]
lemmatized_words = [lemmatizer.lemmatize(word) for word in words]

print("Original Words:", words)
print("Stemmed Words:", stemmed_words)
print("Lemmatized Words:", lemmatized_words)
```

Sample Input:

Enter text: The cats are running faster than the dogs

Sample Output:

Original Words: ['The', 'cats', 'are', 'running', 'faster', 'than', 'the', 'dogs']

Stemmed Words: ['the', 'cat', 'are', 'run', 'faster', 'than', 'the', 'dog']

Lemmatized Words: ['The', 'cat', 'are', 'running', 'faster', 'than', 'the', 'dog']

Experiment No. 13

Objective:

Implement Tic-Tac-Toe game with an AI opponent.

Algorithm (Tic-Tac-Toe with AI Agent):

1. Represent the board as a list of 9 elements.
2. Display the board after each move.
3. Alternate turns between the human player and AI.
4. Validate moves to ensure the selected cell is empty.
5. AI chooses moves using the Minimax algorithm to maximize its chance of winning.
6. After each move, check for a winner or draw.
7. Repeat until the game ends.

Python Code:

```
import math

def print_board(board):
    for i in range(0, 9, 3):
        print(board[i:i+3])
    print()

def check_winner(board, player):
    win_conditions = [
        [0,1,2],[3,4,5],[6,7,8],
        [0,3,6],[1,4,7],[2,5,8],
        [0,4,8],[2,4,6]
    ]
    for condition in win_conditions:
        if all(board[i] == player for i in condition):
            return True
    return False

def available_moves(board):
    return [i for i, x in enumerate(board) if x == '']

def minimax(board, is_maximizing):
    if check_winner(board, 'O'):
        return 1
    if check_winner(board, 'X'):
        return -1
    if '' not in board:
        return 0

    if is_maximizing:
        best_score = -math.inf
        for move in available_moves(board):
            board[move] = 'O'
            score = minimax(board, False)
            board[move] = ''
```



```

        best_score = max(score, best_score)
    return best_score
else:
    best_score = math.inf
    for move in available_moves(board):
        board[move] = 'X'
        score = minimax(board, True)
        board[move] = ' '
        best_score = min(score, best_score)
    return best_score

def ai_move(board):
    best_score = -math.inf
    best_move = None
    for move in available_moves(board):
        board[move] = 'O'
        score = minimax(board, False)
        board[move] = ' '
        if score > best_score:
            best_score = score
            best_move = move
    board[best_move] = 'O'
    print(f"AI plays at position {best_move}")

board = [' '] * 9
print("Positions are 0-8 as follows:")
print_board([i for i in range(9)])

while True:
    print_board(board)
    user_move = int(input("Enter your move (0-8): "))
    if board[user_move] != ' ':
        print("Invalid move, try again.")
        continue
    board[user_move] = 'X'
    if check_winner(board, 'X'):
        print_board(board)
        print("You win!")
        break
    if ' ' not in board:
        print_board(board)
        print("It's a draw!")
        break

    ai_move(board)
    if check_winner(board, 'O'):
        print_board(board)
        print("AI wins!")
        break
    if ' ' not in board:
        print_board(board)
        print("It's a draw!")

```

break

Sample Input:

Enter your move (0-8): 0

Enter your move (0-8): 4

Enter your move (0-8): 8

Sample Output:

AI plays at position 1

AI plays at position 2

['X', 'O', 'O']

['', 'X', '']

['', '', 'X']

You win!

Experiment No. 14

Objective:

Implement Minimax Algorithm for turn-based games

Algorithm (Minimax for Turn-Based Games):

1. Represent the game state and possible moves.
2. Define a function to evaluate the score of terminal states (win, lose, draw).
3. Recursively explore all possible moves for both players:
 - Maximizing player chooses the move with the highest score.
 - Minimizing player chooses the move with the lowest score.
4. Return the optimal move for the AI at each turn.
5. Repeat until the game ends.

Python Code (Example: Simple Turn-Based Game, Tic-Tac-Toe Style):

```
import math

def print_board(board):
    for i in range(0, len(board), 3):
        print(board[i:i+3])
    print()

def check_winner(board, player):
    win_conditions = [
        [0,1,2],[3,4,5],[6,7,8],
        [0,3,6],[1,4,7],[2,5,8],
        [0,4,8],[2,4,6]
    ]
    for condition in win_conditions:
        if all(board[i] == player for i in condition):
            return True
    return False

def available_moves(board):
    return [i for i, x in enumerate(board) if x == ' ']

def minimax(board, is_maximizing):
    if check_winner(board, 'O'):
        return 1
    if check_winner(board, 'X'):
        return -1
    if ' ' not in board:
        return 0

    if is_maximizing:
        best_score = -math.inf
        for move in available_moves(board):
            board[move] = 'O'
            score = minimax(board, False)
            board[move] = ' '
```

```

        best_score = max(score, best_score)
    return best_score
else:
    best_score = math.inf
    for move in available_moves(board):
        board[move] = 'X'
        score = minimax(board, True)
        board[move] = ' '
        best_score = min(score, best_score)
    return best_score

def ai_move(board):
    best_score = -math.inf
    best_move = None
    for move in available_moves(board):
        board[move] = 'O'
        score = minimax(board, False)
        board[move] = ' '
        if score > best_score:
            best_score = score
            best_move = move
    board[best_move] = 'O'
    print(f"AI plays at position {best_move}")

# Initialize board
board = [' '] * 9
print("Positions are 0-8 as follows:")
print_board([i for i in range(9)])

while True:
    print_board(board)
    user_move = int(input("Enter your move (0-8): "))
    if board[user_move] != ' ':
        print("Invalid move, try again.")
        continue
    board[user_move] = 'X'
    if check_winner(board, 'X'):
        print_board(board)
        print("You win!")
        break
    if ' ' not in board:
        print_board(board)
        print("It's a draw!")
        break

    ai_move(board)
    if check_winner(board, 'O'):
        print_board(board)
        print("AI wins!")
        break
    if ' ' not in board:
        print_board(board)

```

```
print("It's a draw!")  
break
```

Sample Input:

Enter your move (0-8): 0
Enter your move (0-8): 4
Enter your move (0-8): 8

Sample Output:

AI plays at position 1
AI plays at position 2
['X', 'O', 'O']
['', 'X', '']
['', '', 'X']
You win!

Experiment No. 15

Objective:

Enhance the game with Alpha-Beta Pruning.

Algorithm (Minimax with Alpha-Beta Pruning):

1. Represent the game board and possible moves.
2. Define a function to evaluate terminal states (win, lose, draw).
3. Use a recursive Minimax function with two additional parameters:
 - Alpha: the best value the maximizer can guarantee.
 - Beta: the best value the minimizer can guarantee.
4. For the maximizing player (AI), update alpha and prune branches where $\alpha \geq \beta$.
5. For the minimizing player (Human), update beta and prune branches where $\beta \leq \alpha$.
6. Select the move with the optimal score for the AI.
7. Repeat until the game ends.

Python Code (Tic-Tac-Toe Example with Alpha-Beta Pruning):

```
import math

def print_board(board):
    for i in range(0, 9, 3):
        print(board[i:i+3])
    print()

def check_winner(board, player):
    win_conditions = [
        [0,1,2],[3,4,5],[6,7,8],
        [0,3,6],[1,4,7],[2,5,8],
        [0,4,8],[2,4,6]
    ]
    for condition in win_conditions:
        if all(board[i] == player for i in condition):
            return True
    return False

def available_moves(board):
    return [i for i, x in enumerate(board) if x == ' ']

def minimax_ab(board, is_maximizing, alpha, beta):
    if check_winner(board, 'O'):
        return 1
    if check_winner(board, 'X'):
        return -1
    if ' ' not in board:
        return 0

    if is_maximizing:
        max_eval = -math.inf
        for move in available_moves(board):
            board[move] = 'O'
```

```

        eval = minimax_ab(board, False, alpha, beta)
        board[move] = ' '
        max_eval = max(max_eval, eval)
        alpha = max(alpha, eval)
        if beta <= alpha:
            break # Beta cut-off
    return max_eval
else:
    min_eval = math.inf
    for move in available_moves(board):
        board[move] = 'X'
        eval = minimax_ab(board, True, alpha, beta)
        board[move] = ' '
        min_eval = min(min_eval, eval)
        beta = min(beta, eval)
        if beta <= alpha:
            break # Alpha cut-off
    return min_eval

def ai_move(board):
    best_score = -math.inf
    best_move = None
    for move in available_moves(board):
        board[move] = 'O'
        score = minimax_ab(board, False, -math.inf, math.inf)
        board[move] = ' '
        if score > best_score:
            best_score = score
            best_move = move
    board[best_move] = 'O'
    print(f"AI plays at position {best_move}")

# Initialize board
board = [' '] * 9
print("Positions are 0-8 as follows:")
print_board([i for i in range(9)])

while True:
    print_board(board)
    user_move = int(input("Enter your move (0-8): "))
    if board[user_move] != ' ':
        print("Invalid move, try again.")
        continue
    board[user_move] = 'X'
    if check_winner(board, 'X'):
        print_board(board)
        print("You win!")
        break
    if ' ' not in board:
        print_board(board)
        print("It's a draw!")
        break

```

```
ai_move(board)
if check_winner(board, 'O'):
    print_board(board)
    print("AI wins!")
    break
if ' ' not in board:
    print_board(board)
    print("It's a draw!")
    break
```

Sample Input:

Enter your move (0-8): 0
Enter your move (0-8): 4
Enter your move (0-8): 8

Sample Output:

AI plays at position 1
AI plays at position 2
['X', 'O', 'O']
[' ', 'X', ' ']
[' ', ' ', 'X']
You win!